	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Aplikacja webowa do zarządzania projektami (realizacja front-endu korzystającego z usługi typu REST)		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	5	Data wykonania	27.06.2026 r.
Ocena		Data oddania spr.	27.07.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie.....	2
2.1. Opis architektury aplikacji webowej	2
2.2. Struktura projektów obu zadań	3
2.3. Konfiguracja połączenia i integracja z REST API.....	5
2.4. Klasy DTO oraz wsparcie dla stronicowania	6
2.5. Warstwa serwisowa, obsługa błędów HTTP	8
2.7. Realizacja zadania obligatoryjnego - obsługa encji Student.....	12
2.8. Realizacja zadania dodatkowego - zarządzanie zadaniami, relacje, wyszukiwanie i stronicowanie	13
3. Wnioski końcowe/Podsumowanie	15

1. Cel laboratorium

Celem tego laboratorium było wykonanie warstwy front-endowej aplikacji webowej przy użyciu frameworka Spring Boot, silnika szablonów Thymeleaf oraz komponentu RestClient. Aplikacja webowa została przygotowana jako interfejs użytkownika, komunikujący się poprzez sieciowe wywołania HTTP/REST z usługą backendową utworzoną podczas laboratorium czwartego (project-rest-api w Eclipse IDE).

W ramach laboratorium zastosowałem rozproszoną architekturę opartą na separacji warstwy prezentacji od warstwy usług i persystencji. Było to związane z konfiguracją klienta REST z obsługą nagłówków uwierzytelniających HTTP Basic Authentication, utworzeniem klas DTO (Data Transfer Object) odizolowanych od JPA/Hibernate, a także zbudowaniem widoków HTML za pomocą silnika Thymeleaf oraz arkuszy stylów CSS.

W laboratorium należało zrealizować obsługę operacji CRUD (Create, Read, Update, Delete) w przeglądarce dla encji Projekt oraz Student (zadanie obligatoryjne), jak również zaimplementować funkcjonalności zarządzające zadaniami i relacjami projektowymi w zadaniu dodatkowym.

2. Rozwiązanie

2.1. Opis architektury aplikacji webowej

Zaimplementowana aplikacja webowa realizuje wzorzec projektowy Model-View-Controller (MVC) uzupełniony o komunikację kliencką z serwerem REST API. Aplikacja jest osobnym procesem serwerowym na porcie 8081 (co skonfigurowałem w pliku application.properties). Umożliwia to równoległe działanie z serwerem back-endu uruchomionym na porcie 8080.

Żądania użytkownika napływające z przeglądarki internetowej są przechwytywane przez kontrolery MVC (oznaczone adnotacją @Controller). Kontrolery nie komunikują się bezpośrednio z bazą danych, ale wywołują metody warstwy serwisowej. Z kolei te ostatnie używają klasy RestClient, formułują i wysyłają sieciowe zapytania HTTP (GET, POST, PUT, DELETE) do backendowej usługi REST. Otrzymana odpowiedź JSON jest następnie przekształcana do postaci obiektu DTO, po czym przekazywana w modelu do szablonu Thymeleaf. Widoki HTML generują ostatecznie odpowiedź, która trafia do przeglądarki klienta.

2.2. Struktura projektów obu zadań

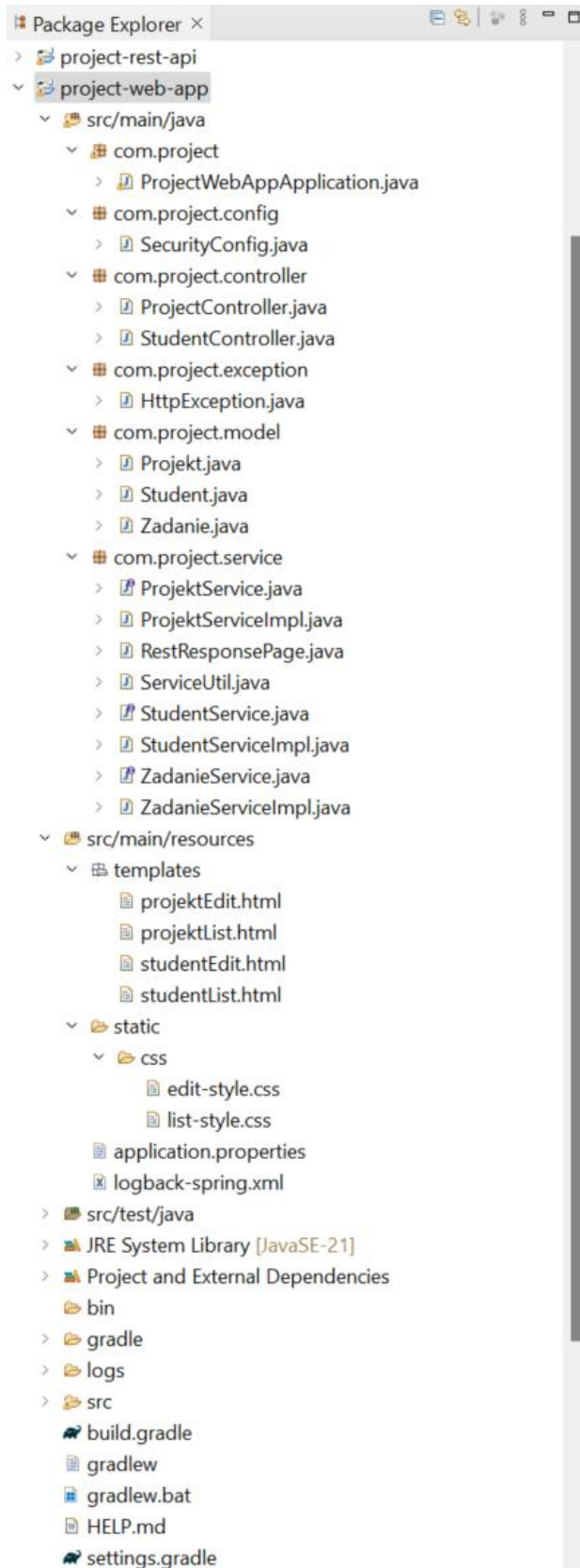
Przygotowałem dwa osobne projekty w środowisku Eclipse IDE: project-web-app dla zadania obligatoryjnego oraz project-web-app-zadanie-dodatkowe dla zadania dodatkowego.

Oba projekty zostały utworzone na bazie narzędzia budowania Gradle. W pliku build.gradle zadeklarowałem w zależnościach m. in. implementacje: 'spring-boot-starter-thymeleaf', 'spring-boot-starter-web', 'spring-boot-starter-validation', moduł Jackson do obsługi typów dat 'jackson-datatype-jsr310' oraz bibliotekę Lombok.

Struktura kodu w przypadku obu projektów zakłada podział na następujące pakiety:

- com.project.config - klasa konfiguracyjna SecurityConfig z definicją komponentu RestClient.
- com.project.controller - kontrolery MVC obsługujące wyznaczanie adresów URL i logikę formularzy.
- com.project.exception - dedykowany wyjątek HttpException do obsługi błędów sieciowych.
- com.project.model - klasy DTO przeznaczone do wymiany danych (Projekt, Zadanie, Student).
- com.project.service - interfejsy oraz klasy implementacyjne serwisów klientów REST, a także klasy pomocnicze RestResponsePage i ServiceUtil.

W katalogu src/main/resources jest podkatalog templates z plikami .html widoków Thymeleaf oraz podkatalog static/css z arkuszami stylów.

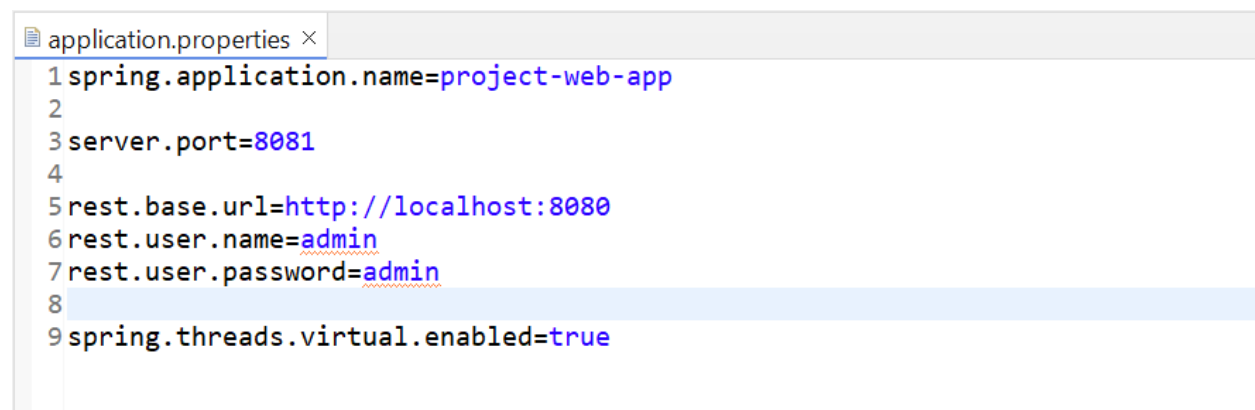


Screen 1 - struktura pakietów i plików projektu webowego w oknie Package Explorer w Eclipse IDE

2.3. Konfiguracja połączenia i integracja z REST API

Komunikacja z zewnętrzną usługą REST wymaga konfiguracji parametrów sieciowych oraz poświadczeń autoryzacyjnych. W pliku `application.properties` zdefiniowałem port aplikacji webowej (`server.port=8081`), adres usługi REST (`rest.base.url=http://localhost:8080`) oraz dane logowania Basic Authentication (`rest.user.name=admin`, `rest.user.password=admin`).

Klasa `SecurityConfig` w pakiecie `com.project.config` jest opatrzona adnotacją `@Configuration` i definiuje ziarno (Bean) typu `RestClient`. Wzorzec projektowy budowniczego `RestClient.builder()` pozwala wstrzyknąć wartości z pliku `application.properties` dzięki adnotacji `@Value`. Do każdego żądania http dołączany jest nagłówek `HttpHeaders.AUTHORIZATION` zawierający ciąg wygenerowany metodą `getBasicAuthenticationHeader()` i umożliwia uwierzytelnianie wszystkich operacji klienta.



```
application.properties x
1 spring.application.name=project-web-app
2
3 server.port=8081
4
5 rest.base.url=http://localhost:8080
6 rest.user.name=admin
7 rest.user.password=admin
8
9 spring.threads.virtual.enabled=true
```

Screen 2 - zawartość pliku `application.properties` z konfiguracją portu 8081 i parametrów REST API

```

SecurityConfig.java x
1 package com.project.config;
2
3 import java.util.Base64;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.context.annotation.Configuration;
7 import org.springframework.http.HttpHeaders;
8 import org.springframework.web.client.RestClient;
9
10 @Configuration
11 public class SecurityConfig {
12     @Value("${rest.base.url}")
13     private String restBaseUrl; // adres wstrzykiwany z pliku application.properties
14
15     @Value("${rest.user.name}")
16     private String restUserName; // nazwa użytkownika wstrzykiwana z pliku application.properties
17
18     @Value("${rest.user.password}")
19     private String restUserPassword; // hasło użytkownika wstrzykiwane z pliku application.properties
20
21     @Bean
22     // dzięki adnotacji @Bean Spring uruchomi metodę i zarejestruje w kontenerze obiekt przez nią zwrócony,
23     // natomiast adnotacja @Autowired i/lub konstruktor użyte w innej klasie spowodują jego wstrzyknięcie
24     public RestClient customRestClient() {
25         return RestClient.builder()
26             .baseUrl(restBaseUrl)
27             .defaultHeader(HttpHeaders.AUTHORIZATION, getBasicAuthenticationHeader(restUserName, restUserPassword))
28             .build();
29     }
30
31     private String getBasicAuthenticationHeader(String username, String password) {
32         return "Basic " + Base64.getEncoder().encodeToString((username + ":" + password).getBytes());
33     }
34 }

```

Screen 3 - klasa konfiguracyjna SecurityConfig.java tworząca ziarno RestClient z nagłówkiem Basic Auth

2.4. Klasy DTO oraz wsparcie dla stronicowania

Względem projektu project-rest-api z poprzedniego laboratorium, z klas modeli skasowałem wszystkie adnotacje relacyjne JPA/Hibernate (np. @Entity, @Table, @Column), skupiając się wyłącznie na obiektach DTO.

W tych klasach jest adnotacja @JsonIgnoreProperties(ignoreUnknown = true), zapobiegająca błędom deserializacji, jeśli usługa REST zwróci pola nieopisane w klasie front-endowej. Pola dat są oprowione adnotacjami @DateTimeFormat, określającymi dokładny wzorzec parsowania ciągu znaków.

Pobieranie list zasobów z usługi REST ze wsparciem dla stronicowania wymagało utworzenia klasy pomocniczej RestResponsePage<T>, która dziedziczy po springowej klasie PageImpl<T>. Ta klasa posiada adnotacje Jacksona (@JsonCreator, @JsonProperty), pozwalające na odczytywanie zwracanego z REST API złożonego obiektu JSON zawierającego tablicę 'content' oraz metadane stronicowania ('pageable', 'totalElements', 'totalPages'). Z kolei pomocnicza

klasa ServiceUtil dostarcza statyczne metody do budowania obiektów URI z parametrami zapytania query (?page=...&size=...&sort=...).

```
Projekt.java x
1 package com.project.model;
2
3 import java.time.LocalDate;
4 import java.time.LocalDateTime;
5 import java.util.List;
6 import java.util.Set;
7 import jakarta.validation.constraints.NotBlank;
8 import jakarta.validation.constraints.Size;
9 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
10 import org.springframework.format.annotation.DateTimeFormat;
11 import lombok.Data;
12 import lombok.NoArgsConstructor;
13 import lombok.AllArgsConstructor;
14 import lombok.Builder;
15
16 @Data
17 @NoArgsConstructor
18 @AllArgsConstructor
19 @Builder
20 @JsonIgnoreProperties(ignoreUnknown = true)
21 public class Projekt {
22
23     private Integer projektId;
24
25     @NotBlank(message = "Pole nazwa nie może być puste!")
26     @Size(min = 3, max = 50, message = "Nazwa musi zawierać od {min} do {max} znaków!")
27     private String nazwa;
28
29     private String opis;
30
31     @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss.SSS")
32     private LocalDateTime dataczasUtworzenia;
33
34     @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss.SSS")
35     private LocalDateTime dataczasModyfikacji;
36
37     @DateTimeFormat(pattern = "yyyy-MM-dd")
38     private LocalDate dataOddania;
39
40     private List<Zadanie> zadania;
41
42     private Set<Student> studenci;
43 }
```

Screen 4 - klasa DTO (np. Projekt.java) z adnotacjami JsonIgnoreProperties i DateTimeFormat

2.5. Warstwa serwisowa, obsługa błędów HTTP

Warstwa serwisowa (w pakiecie `com.project.service`, np. klasa `ProjektServiceImpl`) realizuje operacje na interfejsie `RestClient`. Obiekt `RestClient` jest wstrzykiwany przez konstruktor klasy. Metoda `getProjekt(projekId)` wykonuje żądanie GET pod podany adres URI i odbiera obiekt w body odpowiedzi. W przypadku wystąpienia błędów po stronie serwera REST, metoda `.onStatus(HttpStatusCode::isError, ...)` przechwytuje kod statusu i rzuca wyjątek `HttpException`.

Metoda `setProjekt(projekt)` może zarówno utworzyć nowy obiekt, jak i zmodyfikować istniejący. Jeśli ID projektu nie jest null, wykonywane jest żądanie PUT. W przypadku nowego obiektu wysyłane jest żądanie POST, a po otrzymaniu statusu 201 Created nagłówek odpowiedzi `Location` jest odczytywany, a z niego pobierany nowy adres URI utworzonego zasobu. Następnie wykonuje się końcowe żądanie GET, żeby pobrać zapisany obiekt.

Jest jeszcze metoda `deleteProjekt(projekId)`, która wysyła odpowiednie żądanie DELETE (kasowania projektu).


```

ProjektServiceImpl.java x
51  @Override
52  public Projekt setProjekt(Projekt projekt) {
53      if (projekt.getProjektId() != null) {
54          String resourcePath = getResourcePath(projekt.getProjektId());
55          Logger.info("REQUEST -> PUT {}", resourcePath);
56          restClient
57              .put()
58              .uri(resourcePath)
59              .accept(MediaType.APPLICATION_JSON)
60              .body(projekt)
61              .retrieve()
62              .onStatus(HttpStatusCode::isError, (req, res) -> {
63                  throw new HttpException(res.getStatusCode(), res.getHeaders());
64              })
65              .toBodilessEntity();
66          return projekt;
67      } else {
68          String resourcePath = getResourcePath();
69          Logger.info("REQUEST -> POST {}", resourcePath);
70          ResponseEntity<Void> response = restClient
71              .post()
72              .uri(resourcePath)
73              .accept(MediaType.APPLICATION_JSON)
74              .body(projekt)
75              .retrieve()
76              .onStatus(HttpStatusCode::isError, (req, res) -> {
77                  throw new HttpException(res.getStatusCode(), res.getHeaders());
78              })
79              .toBodilessEntity();
80
81          URI location = response.getHeaders().getLocation();
82          Logger.info("REQUEST (location) -> GET {}", location);
83          return restClient
84              .get()
85              .uri(location)
86              .retrieve()
87              .onStatus(HttpStatusCode::isError, (req, res) -> {
88                  throw new HttpException(res.getStatusCode(), res.getHeaders());
89              })
90              .body(Projekt.class);
91      }
92  }
93
94  @Override
95  public void deleteProjekt(Integer projektId) {
96      String resourcePath = getResourcePath(projektId);
97      Logger.info("REQUEST -> DELETE {}", resourcePath);
98      restClient
99          .delete()
100          .uri(resourcePath)
101          .retrieve()
102          .onStatus(HttpStatusCode::isError, (req, res) -> {
103              throw new HttpException(res.getStatusCode(), res.getHeaders());
104          })
105          .toBodilessEntity();
106  }

```

Screen 5 - kod metod setProjekt i deleteProjekt w klasie ProjektServiceImpl.java wykorzystujący RestClient

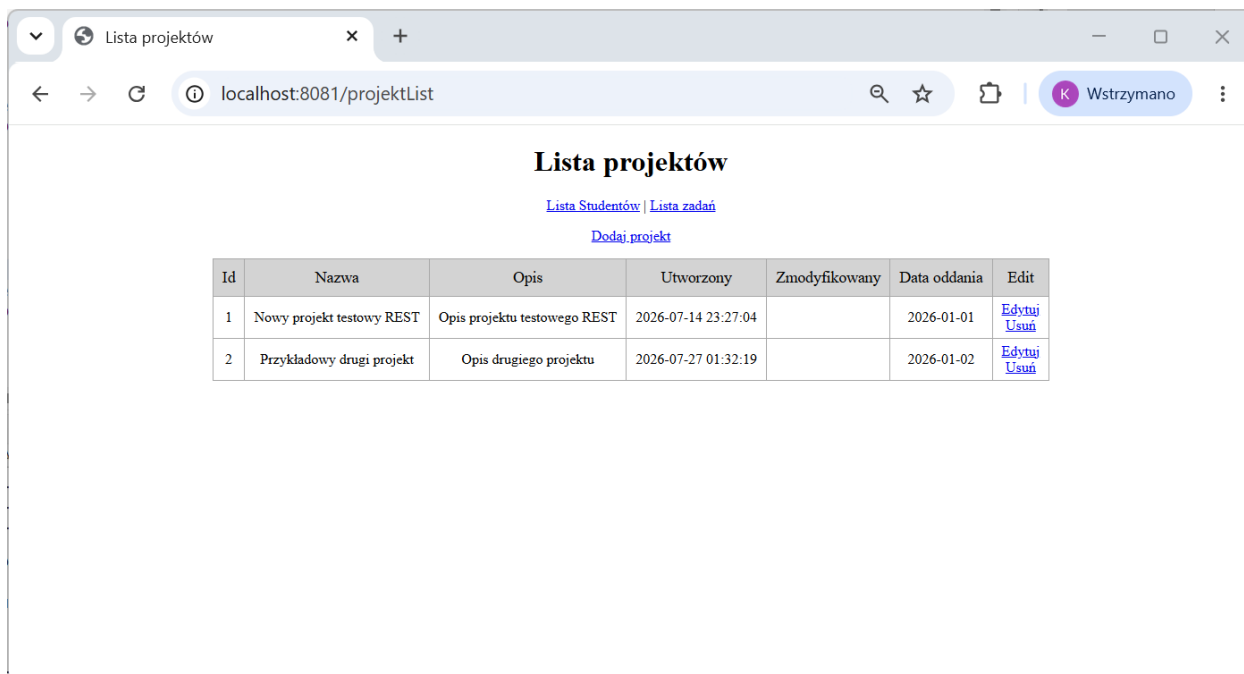
2.6. Kontrolery MVC i szablony widoków Thymeleaf

Kontroler `ProjectController` (oznaczony adnotacją `@Controller`) zarządza ścieżkami w przeglądarce. Metoda `@GetMapping("/projektList")` służy do pobrania strony projektów i dodania jej do obiektu `Model` pod kluczem 'projekty', po czym zwraca nazwę szablonu 'projektList'. Następną metodą - `@GetMapping("/projektEdit")` - przekazuje do formularza 'projektEdit' nowy lub istniejący projekt pobrany na podstawie parametru `id`.

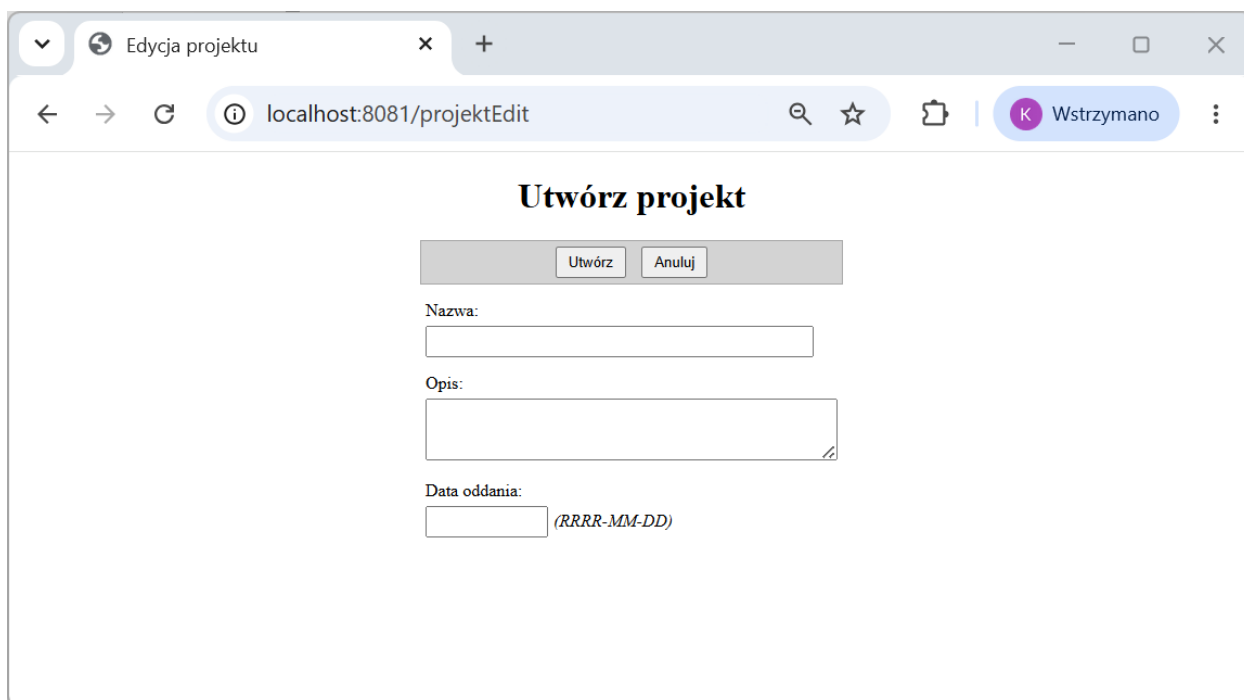
W metodzie `@PostMapping("/projektEdit")` dane z formularza są zapisywane, przy czym parametr '@ModelAttribute @Valid Projekt projekt' weryfikuje poprawność wprowadzanych danych. Z kolei `BindingResult` sprawdza błędy walidacji - jeśli wystąpią, to użytkownik pozostaje na stronie formularza z wyświetlonymi komunikatami błędu. Po pomyślnym zapisie następuje przekierowanie "redirect:/projektList". Widoki Thymeleaf używają `th:each`, `th:field`, `th:action` oraz `th:errors` do renderowania tabel HTML i elementów formularzy.

```
1 package com.project.controller;
2
3 import jakarta.validation.Valid;
4 import org.springframework.data.domain.Pageable;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.ui.Model;
7 import org.springframework.validation.BindingResult;
8 import org.springframework.web.bind.annotation.GetMapping;
9 import org.springframework.web.bind.annotation.ModelAttribute;
10 import org.springframework.web.bind.annotation.PostMapping;
11 import org.springframework.web.bind.annotation.RequestParam;
12 import org.springframework.web.client.HttpStatusCodeException;
13 import com.project.model.Projekt;
14 import com.project.service.ProjektService;
15
16 @Controller
17 public class ProjectController {
18     private final ProjektService projektService;
19
20     public ProjectController(ProjektService projektService) {
21         this.projektService = projektService;
22     }
23
24     @GetMapping("/projektList")
25     public String projektList(Model model, Pageable pageable) {
26         model.addAttribute("projekty", projektService.getProjekty(pageable).getContent());
27         return "projektList";
28     }
29
30     @GetMapping("/projektEdit")
31     public String projektEdit(@RequestParam(name="projectId", required=false) Integer projectId, Model model) {
32         if (projectId != null) {
33             model.addAttribute("projekt", projektService.getProjekt(projectId).orElse(new Projekt()));
34         } else {
35             model.addAttribute("projekt", new Projekt());
36         }
37         return "projektEdit";
38     }
39
40     @PostMapping(path = "/projektEdit")
41     public String projektEditSave(@ModelAttribute @Valid Projekt projekt, BindingResult bindingResult) {
42         if (bindingResult.hasErrors()) {
43             return "projektEdit";
44         }
45         try {
46             projektService.setProjekt(projekt);
47         } catch (HttpStatusCodeException e) {
48             bindingResult.rejectValue("", String.valueOf(e.getStatusCode().value()), e.getStatusCode().toString());
49             return "projektEdit";
50         }
51         return "redirect:/projektList";
52     }
53
54     @PostMapping(params = "cancel", path = "/projektEdit")
55     public String projektEditCancel() {
56         return "redirect:/projektList";
57     }
58
59     @PostMapping(params = "delete", path = "/projektEdit")
60     public String projektEditDelete(@ModelAttribute Projekt projekt) {
61         projektService.deleteProjekt(projekt.getProjectId());
62         return "redirect:/projektList";
63     }
64 }
```

Screen 6 - kod kontrolera `ProjectController.java` z obsługą widoków i walidacji `BindingResult`



Screen 7 - widok listy projektów (projektList.html) uruchomiony w przeglądarce pod adresem <http://localhost:8081/projektList> (po wcześniejszym uruchomieniu serwera REST oraz aplikacji webowej w Eclipse IDE)

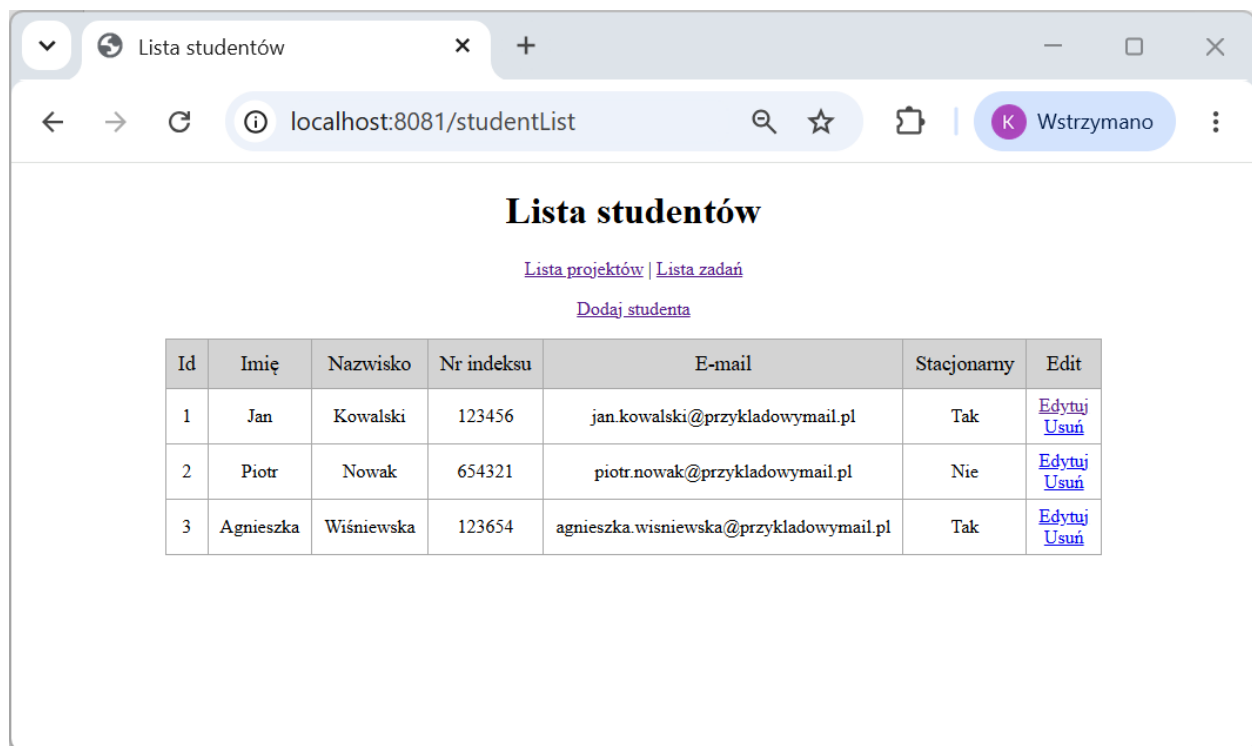


Screen 8 - formularz tworzenia projektu (projektEdit.html) w przeglądarce internetowej

2.7. Realizacja zadania obligatoryjnego - obsługa encji Student

Zgodnie z wymaganiami zadania obligatoryjnego z instrukcji laboratoryjnej, rozszerzyłem funkcjonalność aplikacji webowej o możliwość dodawania, edytowania i usuwania danych studentów. W tym celu w pakiecie `com.project.model` utworzyłem klasę `Student.java`, zawierającą pola: `studentId`, imię, nazwisko, `nrIndeksu`, email oraz `stacjonarny`.

W `com.project.service` utworzyłem interfejs `StudentService` oraz klasę `StudentServiceImpl` z `RestClient` do komunikacji z endpointem REST `/api/studenci`. W następnym kroku dodałem kontroler `StudentController` oraz dwa szablony Thymeleaf: `studentList.html` (czyli tabela z wykazem studentów oraz z przyciskami Edytuj i Usuń) oraz `studentEdit.html` (formularz wprowadzania danych studenta). Pomiędzy widokiem projektów a widokiem studentów występuje pełna nawigacja.



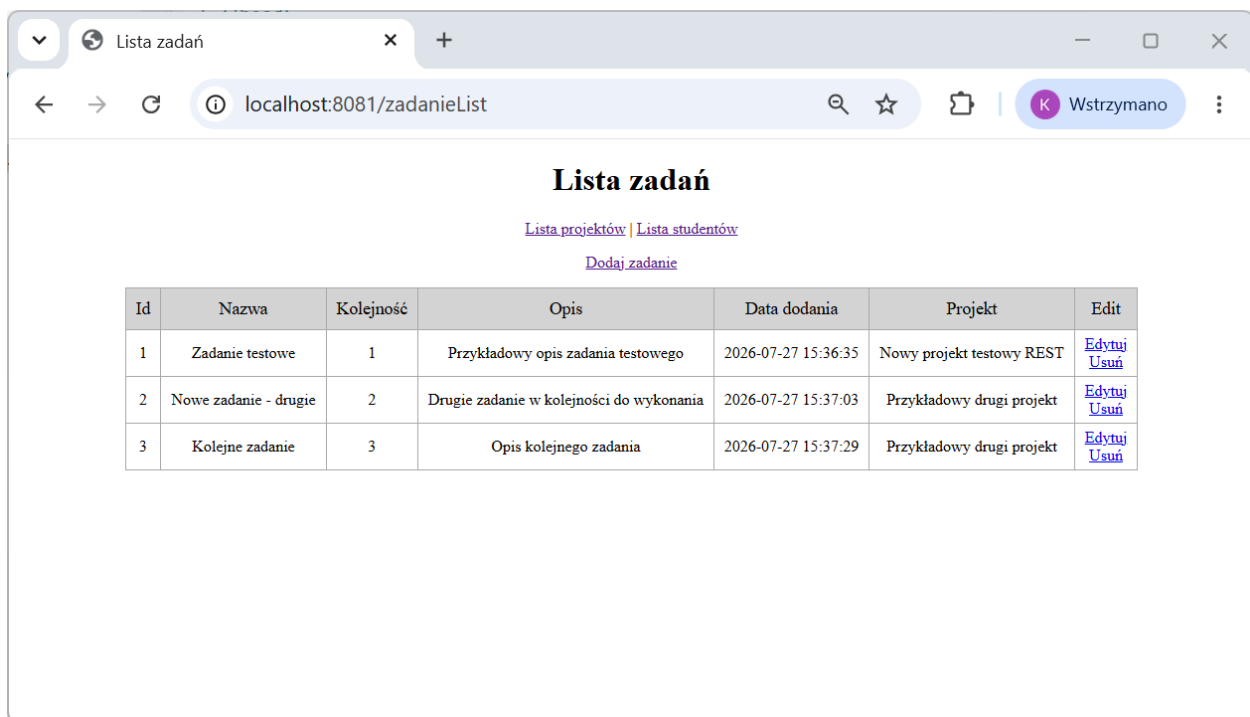
Id	Imię	Nazwisko	Nr indeksu	E-mail	Stacjonarny	Edit
1	Jan	Kowalski	123456	jan.kowalski@przykladowymail.pl	Tak	Edytuj Usuń
2	Piotr	Nowak	654321	piotr.nowak@przykladowymail.pl	Nie	Edytuj Usuń
3	Agnieszka	Wiśniewska	123654	agnieszka.wisniewska@przykladowymail.pl	Tak	Edytuj Usuń

Screen 9 - widok listy studentów (`studentList.html`) zrealizowany w ramach zadania obligatoryjnego

2.8. Realizacja zadania dodatkowego - zarządzanie zadaniami, relacje, wyszukiwanie i stronicowanie

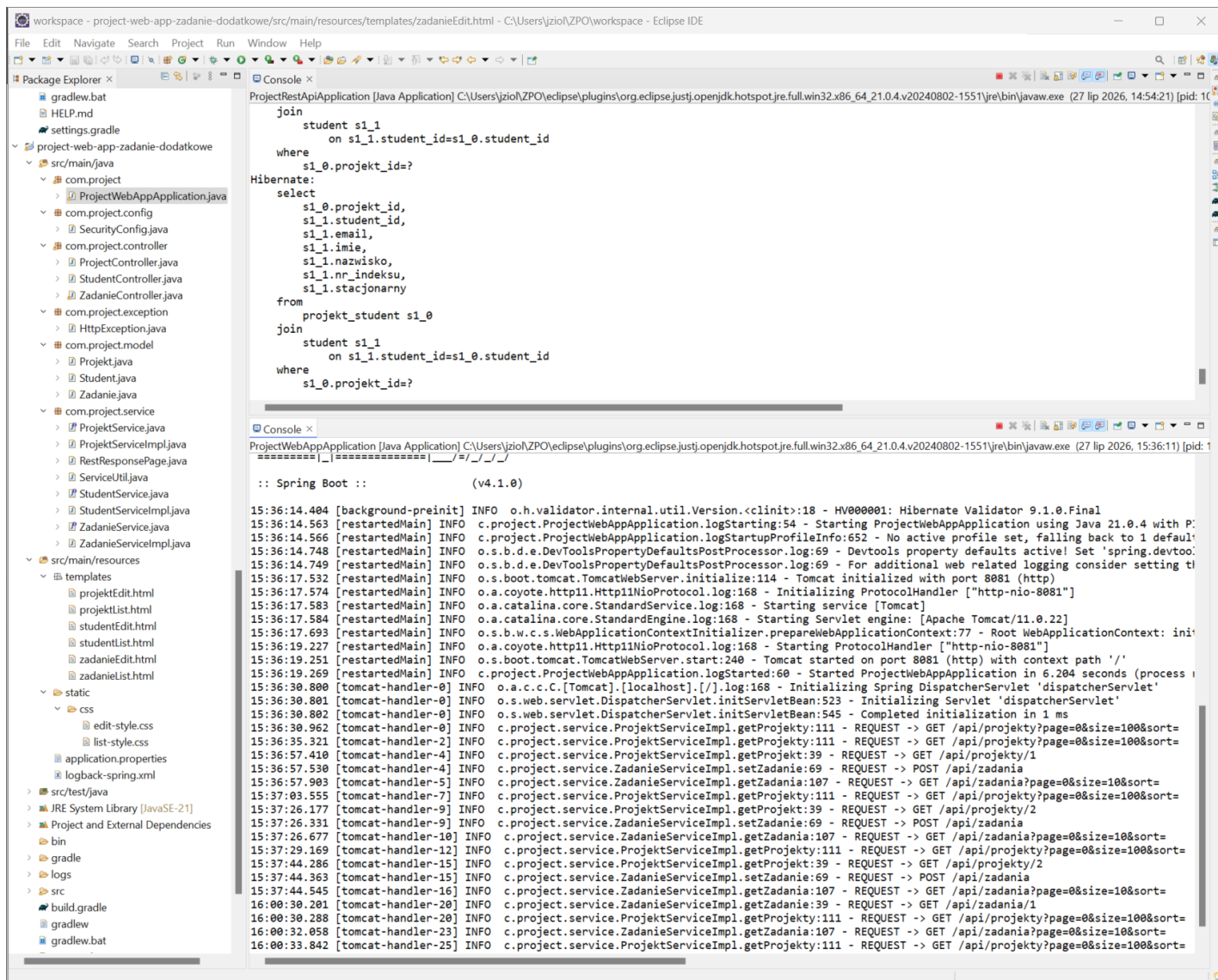
W projekcie `project-web-app-zadanie-dodatkowe` zaimplementowałem funkcjonalności zarządzania zadaniami tj. dodawanie, modyfikowanie i usuwanie zadań w systemie. Zadania projektowe są zarządzane przez połączenie klas: `Zadanie`, `ZadanieService`, `ZadanieServiceImpl` oraz `ZadanieController`. W zadaniu dodatkowym w katalogu `src/main/resources/templates` dodałem widoki `zadanieList.html` oraz `zadanieEdit.html`, ewidencjonujące zadania przypisane do danego projektu.

Ponadto, w tym zadaniu rozszerzyłem system o przypisywanie studentów oraz zadań do konkretnych projektów poprzez relacyjne formularze. Dla listy studentów zaimplementowałem też mechanizmy stronicowania (paski nawigacyjne paginacji: przejście do pierwszej, poprzedniej, następnej i ostatniej strony, wybór rozmiaru strony) i wyszukiwania (pola wyszukiwania tekstowego filtrujące dane w czasie rzeczywistym), które ułatwią dostęp do danych w systemie.



Lista zadań						
Lista projektów Lista studentów						
Dodaj zadanie						
Id	Nazwa	Kolejność	Opis	Data dodania	Projekt	Edit
1	Zadanie testowe	1	Przykładowy opis zadania testowego	2026-07-27 15:36:35	Nowy projekt testowy REST	Edytuj Usuń
2	Nowe zadanie - drugie	2	Drugie zadanie w kolejności do wykonania	2026-07-27 15:37:03	Przykładowy drugi projekt	Edytuj Usuń
3	Kolejne zadanie	3	Opis kolejnego zadania	2026-07-27 15:37:29	Przykładowy drugi projekt	Edytuj Usuń

Screen 10 - widok listy zadań (przypisanych do konkretnych projektów) w zadaniu dodatkowym



Screen 11 - podzielone okno konsoli w środowisku Eclipse IDE wyświetlające logi zapytań SQL backendu (ProjectRestApiApplication) oraz logi wywołań żądań HTTP front-endu (ProjectWebAppApplication)

3. Wnioski końcowe/Podsumowanie

W piątym laboratorium bezpośrednio kontynuowałem prace podjęte w ramach laboratorium czwartego. Podczas gdy poprzednie laboratorium zakładało budowę usługi backendowej REST API, obecne zadanie polegało na zaimplementowaniu warstwy prezentacji (Front-end MVC) wykorzystującej framework Spring Boot oraz silnik szablonów Thymeleaf.

W architekturze nowej warstwy zastosowałem komponent RestClient. Ten element umożliwił formułowanie żądań HTTP z dołączaniem nagłówków uwierzytelniających HTTP Basic Authentication oraz obsługą błędów sieciowych poprzez wyjątek `HttpException`. Odseparowałem też całkowicie klasy DTO na front-endlu od klas encyjnych JPA wykorzystywanych w backendzie. Umożliwiło to ich niezależne rozwijanie bez ryzyka przenoszenia zależności bazodanowych na interfejs użytkownika.

Wykorzystanie Thymeleaf w połączeniu z mechanizmami Spring Bean Validation i obiektem `BindingResult` sprawiło, że mogłem szybko zbudować dynamiczne widoki HTML oraz dodać obsługę walidacji formularzy nie musząc pisać złożonych skryptów w JavaScript. Zrealizowanie operacji CRUD dla danych studentów w zadaniu obligatoryjnym oraz zadań wraz z przypisywaniem, stronicowaniem i wyszukiwaniem w zadaniu dodatkowym, potwierdza poprawność opisywanej architektury.

Podsumowując, zaimplementowałem aplikację webową tworzącą, wraz z back-endem REST API, kompletny system typu klient-serwer. Wszystkie założenia i cele wskazane w instrukcji laboratoryjnej zostały osiągnięte.